Learning Report – Module 13: Agile Methodology

Overview

This module introduced me to the fundamentals of Agile Methodology, a widely used approach for software development that focuses on collaboration, flexibility, and continuous improvement. I learned about the Agile Manifesto, Scrum framework, Agile estimation techniques, sprint planning, user stories, and acceptance criteria. The module emphasized delivering software in small increments while adapting to changing requirements.

Learning Outcomes

After completing this module, I was able to:

- Understand the principles and values of Agile.
- Differentiate between Agile and Waterfall methodologies.
- Explain the Scrum framework, including roles, ceremonies, and artifacts.
- Understand Agile estimation techniques such as story points and planning poker.
- Interpret velocity and burndown charts.
- Learn the sprint planning process.
- Write effective user stories using the INVEST principle.
- Create acceptance criteria using the Given-When-Then format.
- Understand teamwork and collaboration in Agile projects.

Introduction to Agile

Agile is a software development methodology that focuses on delivering high-quality software through continuous collaboration, customer feedback, and iterative development. Instead of completing the entire project at once, Agile divides the work into smaller iterations called Sprints, allowing teams to adapt to changing requirements more effectively.

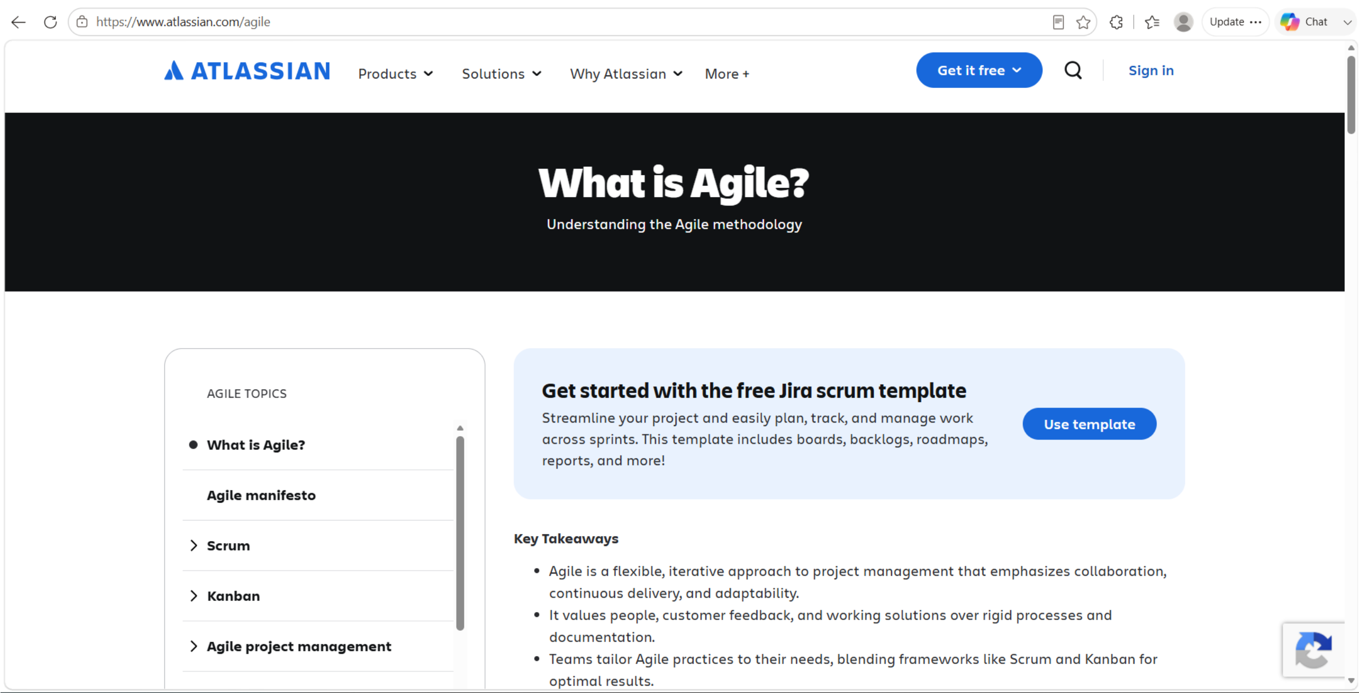
Agile Manifesto

The Agile Manifesto is based on four core values:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.

- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

It also includes twelve principles that promote customer satisfaction, continuous improvement, teamwork, and regular software delivery.



Agile vs Waterfall

Agile is flexible and supports continuous changes throughout the project, whereas the Waterfall model follows a sequential approach where each phase must be completed before moving to the next. Agile encourages frequent customer feedback and iterative development, making it more suitable for modern software projects.

Scrum Framework

Scrum is one of the most popular Agile frameworks used to manage software development projects.

Scrum Roles

- **Product Owner** – Defines project requirements and prioritizes the product backlog.
- **Scrum Master** – Facilitates Scrum activities and removes obstacles for the team.
- **Development Team** – Designs, develops, tests, and delivers the product increment.

Scrum Ceremonies

The Scrum framework includes several important events:

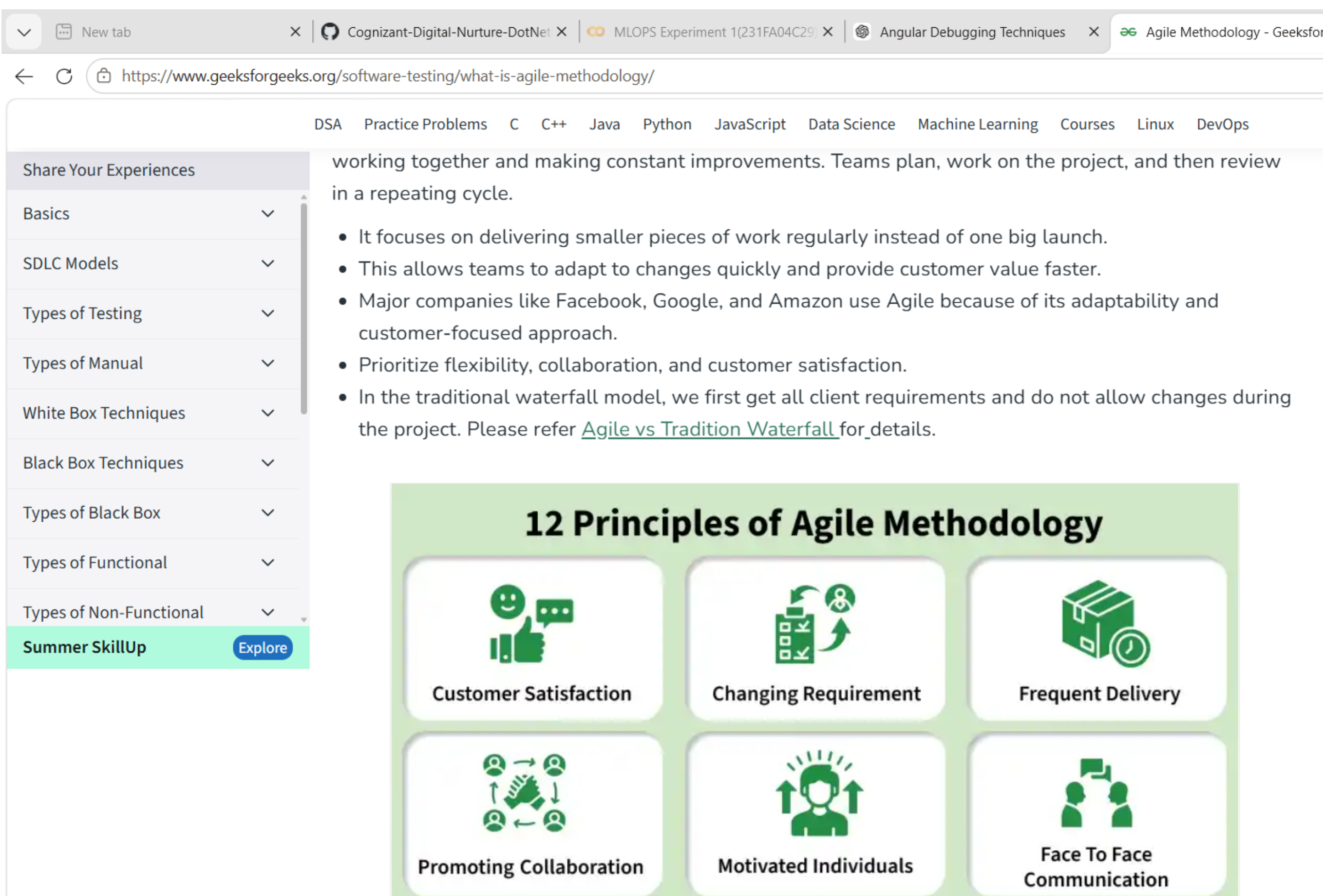
- **Sprint Planning** – Plans the work for the upcoming sprint.
- **Daily Scrum** – A short daily meeting to discuss progress and challenges.
- **Sprint Review** – Demonstrates completed work to stakeholders.
- **Sprint Retrospective** – Reviews the sprint and identifies improvements for future iterations.

Scrum Artifacts

The main Scrum artifacts are:

- Product Backlog
- Sprint Backlog
- Product Increment

The Definition of Done (DoD) ensures that completed work meets the agreed quality standards before it is considered finished.



Agile Estimation and Planning

Agile teams estimate work using Story Points, which measure the complexity and effort required to complete a task instead of estimating time.

Planning Poker

Planning Poker is an estimation technique where team members independently estimate story points using numbered cards. After discussion, the team agrees on a final estimate.

Sprint Planning

Sprint planning involves selecting user stories from the product backlog, estimating the work, assigning tasks, and defining the sprint goal.

Velocity and Burndown Charts

- **Velocity** measures the amount of work completed during a sprint.
- **Burndown Chart** shows the remaining work throughout the sprint and helps track progress.

These metrics help teams improve planning and monitor project progress.

Agile User Stories

A User Story describes a software feature from the user's perspective.

The standard format is:

As a user, I want a feature, so that I receive a benefit.

Example:

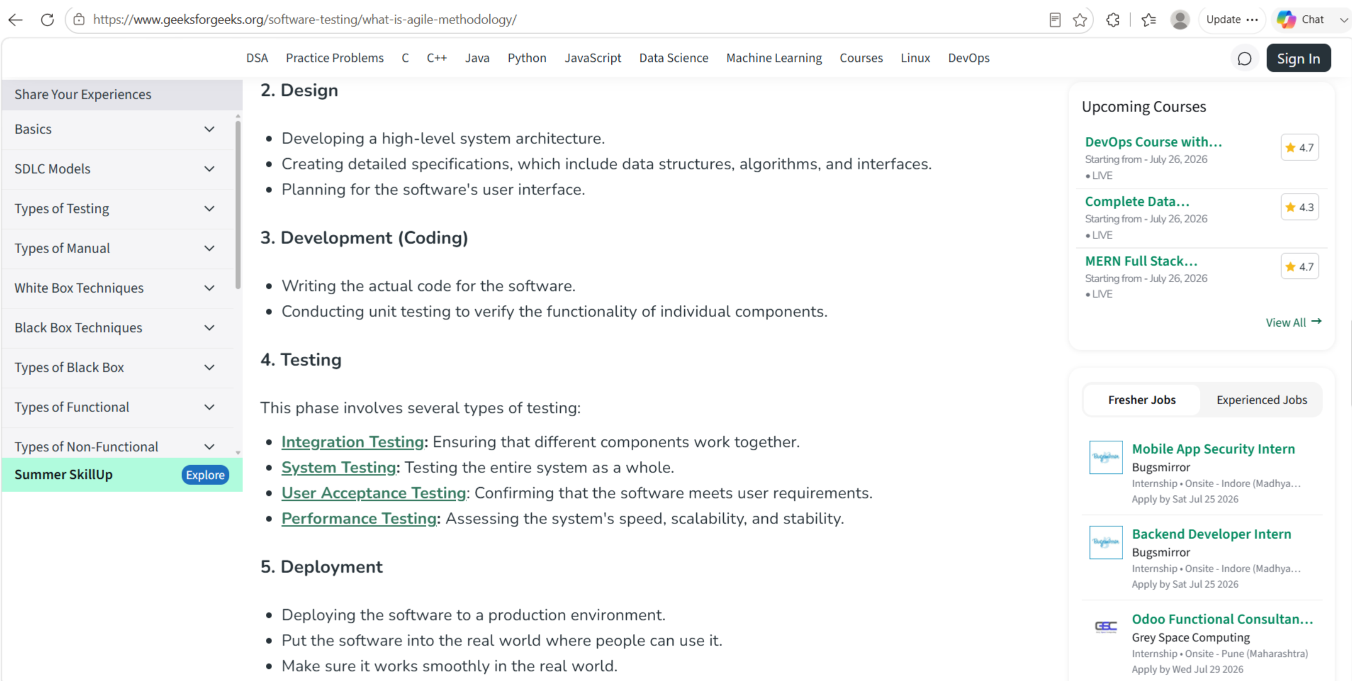
As a student, I want to view my attendance report so that I can track my attendance percentage.

INVEST Principle

A good user story should be:

- **Independent**
- **Negotiable**
- **Valuable**
- **Estimable**
- **Small**

- **Testable**



Acceptance Criteria

Acceptance criteria define the conditions that must be satisfied before a user story is considered complete.

A common format is:

- **Given** a specific condition,
- **When** an action is performed,
- **Then** the expected result should occur.

This format helps developers and testers clearly understand the expected behavior of a feature.

Conclusion

This module provided a strong foundation in Agile methodology and its practical application in software development. I learned the Agile principles, Scrum framework, estimation techniques, sprint planning process, and the importance of user stories and acceptance criteria. I also understood how Agile promotes collaboration, continuous feedback, and incremental software delivery. Overall, this module improved my understanding of Agile practices and prepared me to work effectively in Agile development teams.